

PENETRATION TESTING REPORT

OWASP Juice Shop

Security Assessment

Client :	Kali Linux Lab Environment
Application :	OWASP Juice Shop (v15.x)
URL :	https://juice-shop.herokuapp.com/
Assessment Period :	April 30, 2026
Report Date :	April 30, 2026
Prepared By :	Ranjana Jadhav
Institution :	CampusPe
Mentor :	Jacob Dennis
Classification :	CONFIDENTIAL

CONFIDENTIALITY NOTICE: This document contains sensitive security information and is intended solely for authorized personnel. Unauthorized disclosure, copying, or distribution of this document is strictly prohibited.

1. EXECUTIVE SUMMARY

Assessment Overview:

A comprehensive security assessment was conducted on the OWASP Juice Shop web application hosted on a local Kali Linux lab environment. The assessment followed the standard penetration testing methodology covering Reconnaissance, Scanning, Vulnerability Assessment, and Exploitation phases.

Overall Risk Level: **CRITICAL**

The assessment identified **9 distinct vulnerabilities** across multiple severity levels. The presence of **1 Critical and 2 High severity vulnerabilities** poses immediate risk. These vulnerabilities could allow an unauthorized attacker to:

- Bypass authentication and gain admin access without a password
- Steal session tokens and impersonate any logged-in user
- Perform unlimited login attempts without any account lockout
- Obtain sensitive server information through verbose error messages

Key Findings Summary :

Severity	Count	CVSS Range	Priority
Critical	1	9.0 - 10.0	P0 - Immediate
High	2	7.0 - 8.9	P1 - Urgent
Medium	4	4.0 - 6.9	P2 - Important
Low	2	0.1 - 3.9	P3 - Standard
Total	9	-	-

Critical Recommendations :

- **Immediate (0-24 hours):** Fix SQL Injection using parameterized queries. Disable verbose error messages.
- **Short-term (1-7 days):** Implement XSS output encoding and account lockout after 5 failed attempts.
- **Medium-term (1-4 weeks):** Add CSP headers, HSTS headers, and cache control fixes.
- **Long-term (1-3 months):** Conduct regular security assessments and implement secure development practices.

3. SCOPE AND METHODOLOGY

3.1 Testing Scope

In-Scope Components:

- **SQL Injection** — Login authentication and data retrieval
- **Cross-Site Scripting (XSS)** — Reflected, Stored, DOM-based
- **Brute Force** — Authentication rate limiting controls
- **File Upload** — File handling and validation mechanisms
- **Information Disclosure** — Error message handling
- **Security Headers** — CSP, HSTS, Cache Control

Out-of-Scope:

- Host operating system vulnerabilities
- Network infrastructure and switching equipment
- Denial of Service (DoS) testing
- Social engineering attacks
- Any application other than OWASP Juice Shop

3.2 Testing Methodology

The assessment followed the industry-standard OWASP Testing Guide methodology and included the following phases:

Phases	Activity	Duration
Reconnaissance	• Port scanning • service enumeration • technology identification	1 hour
Scanning	• Automated vulnerability scanning against <code>http://localhost:3000</code>	1 hour
Vulnerability Assessment	• Burp Suite • Firefox Developer Tools	2 hour
Exploitation	• Browser • Manual Payloads	1 hour
Reporting	• Documentation, • evidence compilation • remediation guidance	2 hour

3.3 Tools Used

- **Nmap 7.98** — Network reconnaissance and port scanning
- **OWASP ZAP 2.17.0** — Automated vulnerability scanning
- **Burp Suite Community Edition** — Web traffic interception
- **Firefox Developer Tools** — Request/response analysis
- **Kali Linux 2025.4** — Primary testing platform
- **CVSS v3.1 Calculator (first.org)** — Severity scoring

4. Findings Summary Table

Vulnerability Priority	Severity	CVSS	CWE	Location
SQL Injection	Critical	9.8	CWE-89	/login — email
Reflected XSS — Cookie Theft	High	8.2	CWE-79	/search — search
Brute Force — No Lockout	High	7.5	CWE-307	/login — password field
Information Disclosure	Medium	5.3	CWE-209	/profile — image upload
CSP Header Missing	Medium	5.4	CWE-693	Application-wide
Cross Domain Misconfigurat ion	Medium	5.3	CWE-942	Application-wide
HSTS Header not set	Medium	4.8	CWE-319	Application-wide
Timestamp Disclosure	Low	3.1	CWE-200	Application-wide

5. DETAILED FINDINGS

5.1 CRITICAL FINDINGS

Finding 1: SQL Injection — Authentication Bypass

Severity: **CRITICAL**

CVSS Score: 9.8

CVSS Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CWE: CWE-89: SQL Injection

Location: <http://localhost:3000/#/login>

Parameter: Email field

Discovery: April 30, 2026

Description:

The login page fails to sanitize user input in the email field before incorporating it into the database query. This allows an attacker to manipulate the SQL query logic and bypass authentication completely without knowing any valid credentials.

Proof of Concept:

Payload: admin'OR'1'='1'--

Field: Email input field

Result: Successfully logged in as admin without password Browser prompted to save credentials showing the injected payload as username, confirming full authentication bypass.

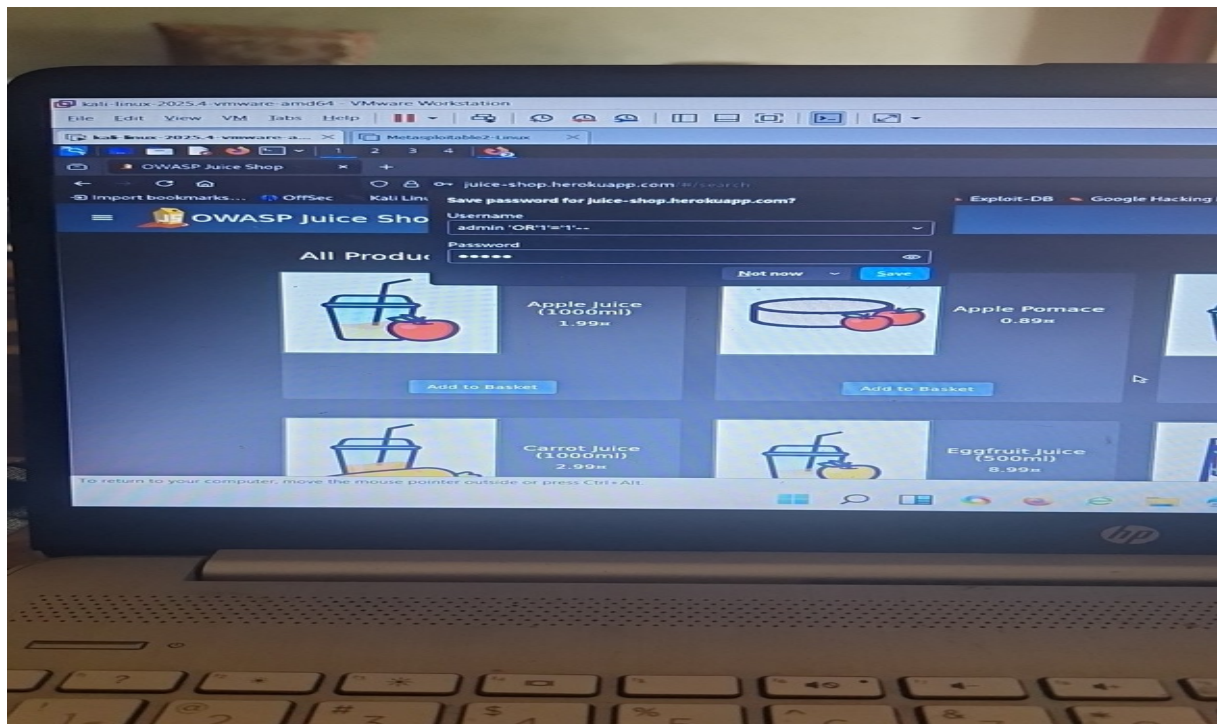


Figure 1: SQL Injection — Admin login bypass (admin OR 1=1)

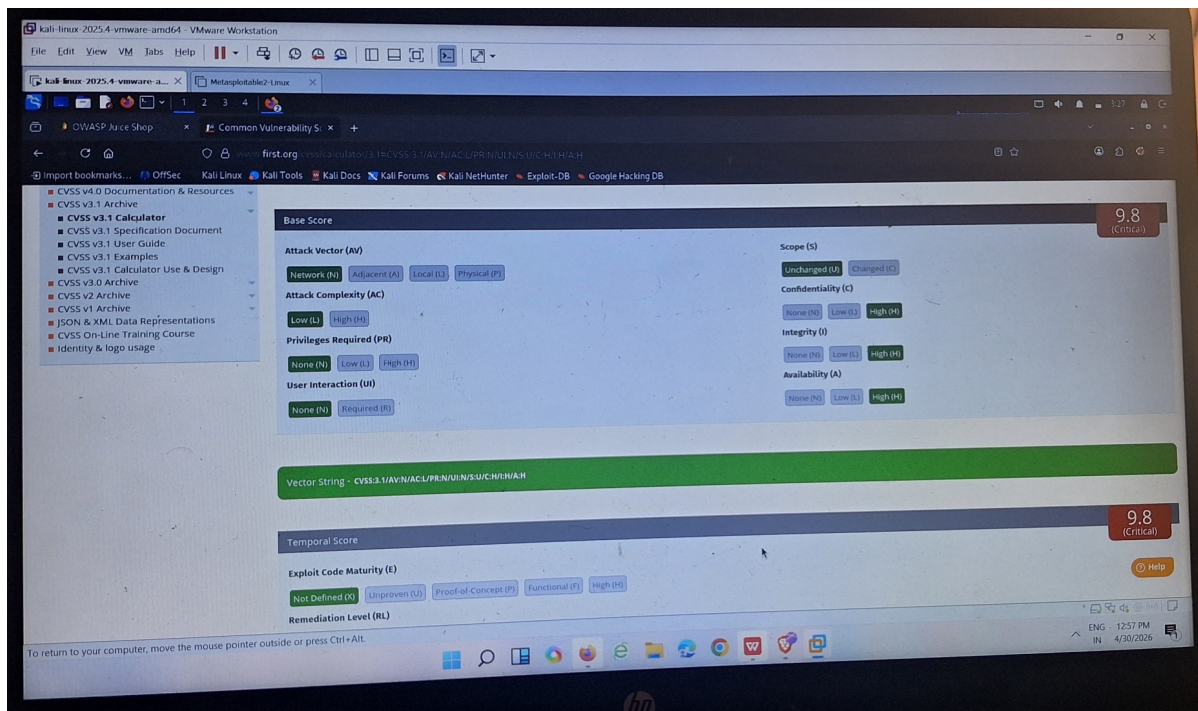


Figure 2: CVSS v3.1 Score 9.8 Critical — SQL Injection

Impact:

Confidentiality: HIGH — Admin account accessed, all user data and credentials exposed

Integrity: HIGH — Attacker can modify or delete database records

Availability: HIGH — Database can be corrupted or dropped

Business Impact:

- Complete authentication bypass without valid credentials
- Access to all user accounts and sensitive data
- Regulatory violations — GDPR, PCI-DSS
- Reputational damage from data breach

Remediation:**Immediate (0-24 hours):**

- Deploy input validation to reject special characters
- Enable database query logging for monitoring

5.2 HIGH SEVERITY FINDINGS

Finding 2: Reflected XSS — Session Token Theft

Severity: HIGH

CVSS Score: 8.2

CVSS Vector: AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N

CWE: CWE-79: Cross-Site Scripting

Location: <https://juice-shop.herokuapp.com/#/search>

Parameter: search bar — q parameter

Discovery: April 30, 2026

Description:

The search bar reflects user input directly into the page without sanitization or output encoding. This allows an attacker to inject JavaScript that executes in the victim's browser context, stealing session tokens and cookies.

Proof of Concept:

Payload:

Field: Search bar

Result:

- Browser executed the injected JavaScript and displayed the full session cookie containing:
 - Admin JWT token
 - Admin email: admin@juice-sh.op
 - Password hash
 - User role: admin

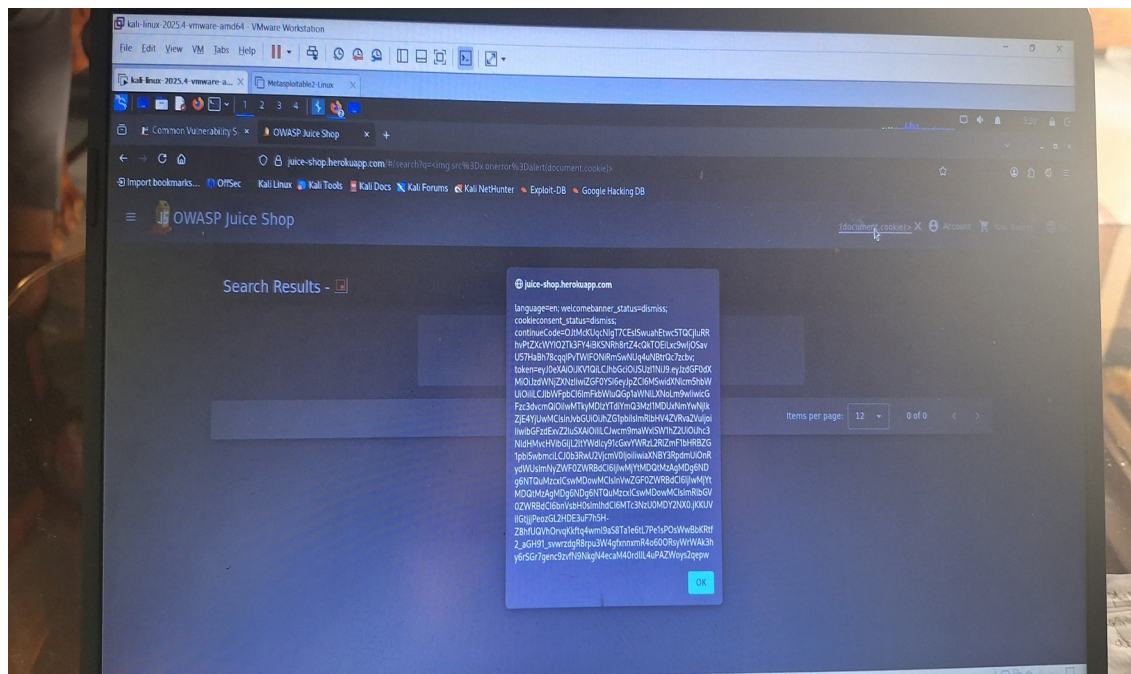


Figure 3: XSS — document.cookie payload exposing JWT token and admin credentials

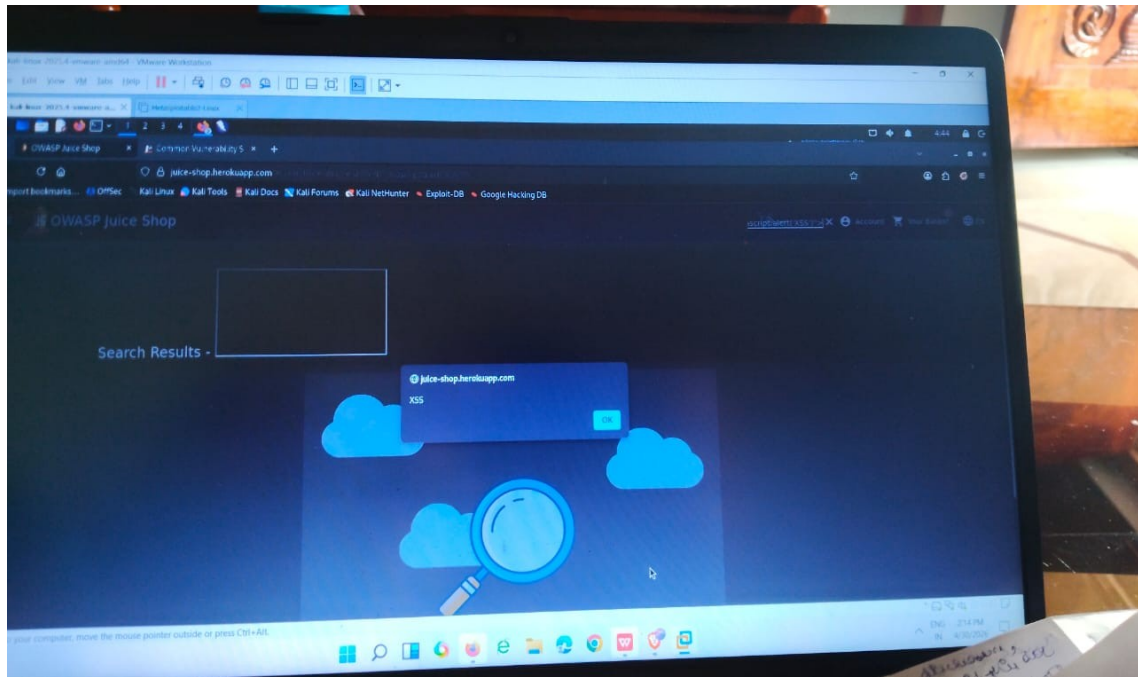


Figure 4: XSS — Basic alert popup confirming script execution

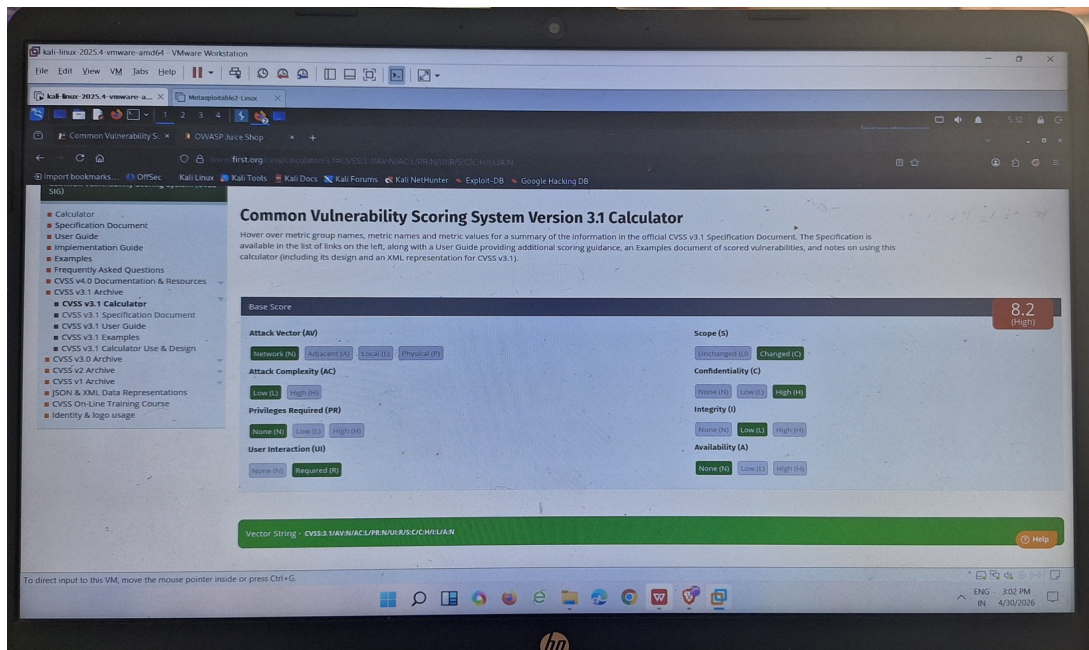


Figure 5: CVSS v3.1 Score 8.2 High — Reflected XSS

Impact:

Confidentiality: **HIGH** — Admin JWT token and all session cookies fully exposed

Integrity: LOW — Page content can be manipulated

Availability: NONE — No service disruption

Business Impact:

An attacker capturing the JWT token can silently impersonate the admin user without the victim's knowledge, gaining full administrative access.

Remediation:**Immediate:**

- Implement output encoding on all user-supplied input
- Add X-XSS-Protection header

Finding 3: Brute Force — Missing Account Lockout

Severity: **HIGH**

CVSS Score: 7.5

CVSS Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N

CWE: CWE-307: Improper Restriction of Authentication Attempts

Location: <https://juice-shop.herokuapp.com/#/login>

Parameter: Password field

Discovery: April 30, 2026

Description:

The application does not implement any account lockout mechanism, CAPTCHA, or rate limiting on the login page. An attacker can make unlimited login attempts without any restriction, enabling fully automated password attacks.

Proof of Concept:

Attempts made:

- admin@juice-sh.op : password1 → Invalid email or password
- admin@juice-sh.op : password2 → Invalid email or password
- admin@juice-sh.op : password3 → Invalid email or password
- admin@juice-sh.op : password4 → Invalid email or password
- admin@juice-sh.op : password5 → Invalid email or password

Result: No lockout triggered, no CAPTCHA shown, no rate limiting applied after 5+ failed attempts.

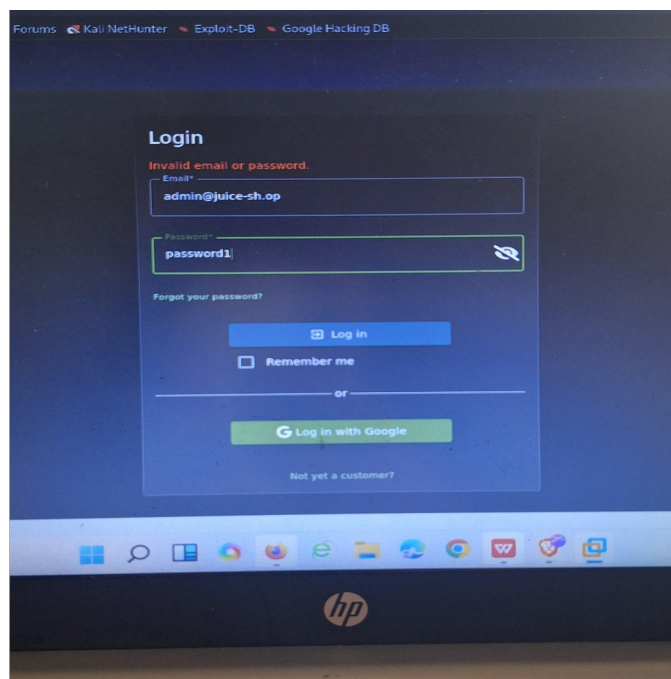


Figure 6: Brute Force — Attempt 1 (password1) — Invalid email or password

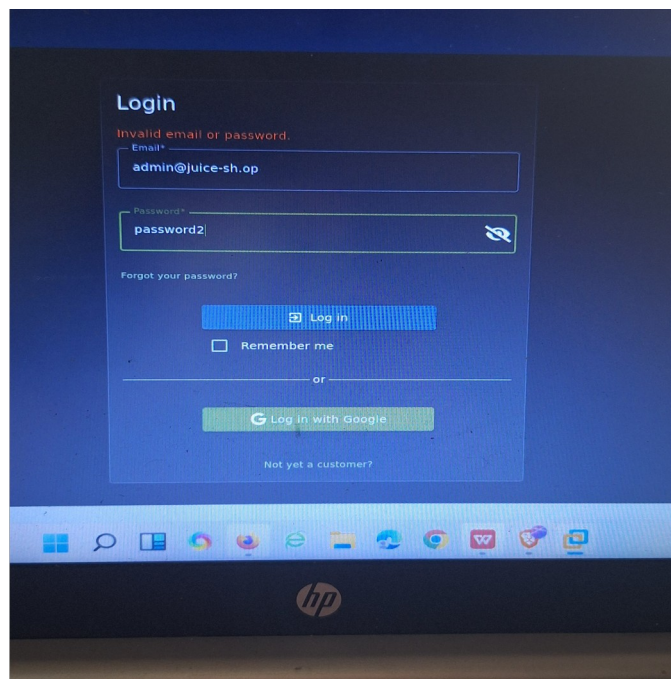


Figure 7: Brute Force — Attempt 2 (password2) — No lockout

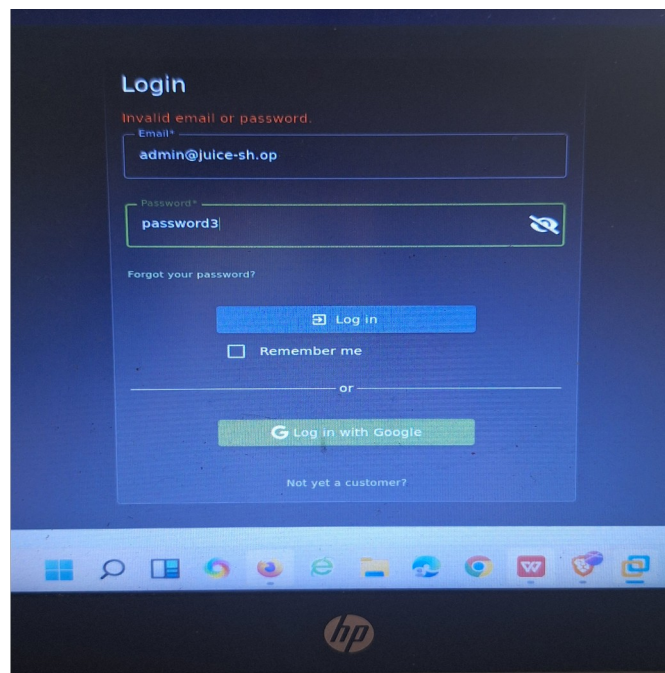


Figure 8: Brute Force — Attempt 3 (password3) — No lockout

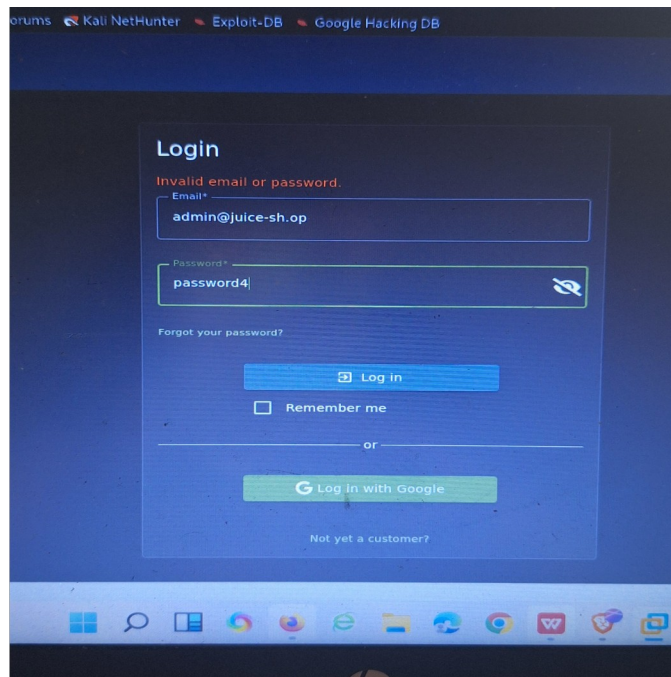


Figure 9: Brute Force — Attempt 4 (password4) — No logout

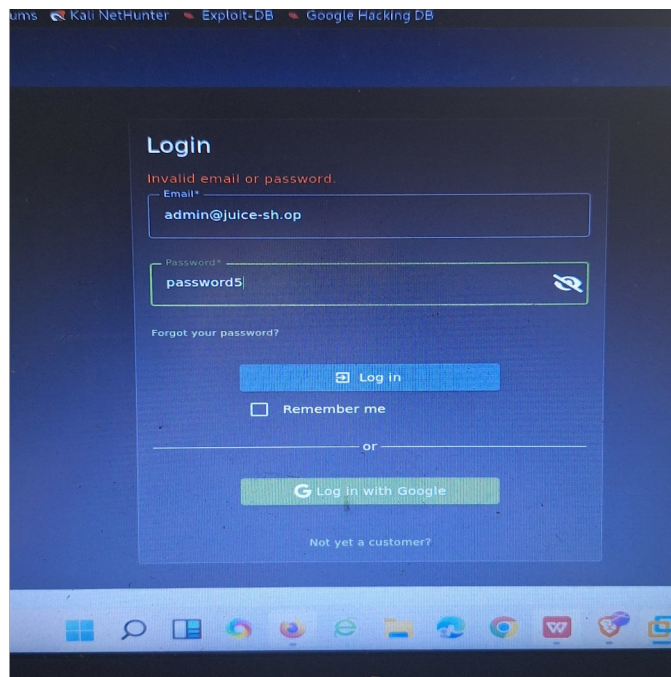


Figure 10: Brute Force — Attempt 5 (password5) — No logout triggered

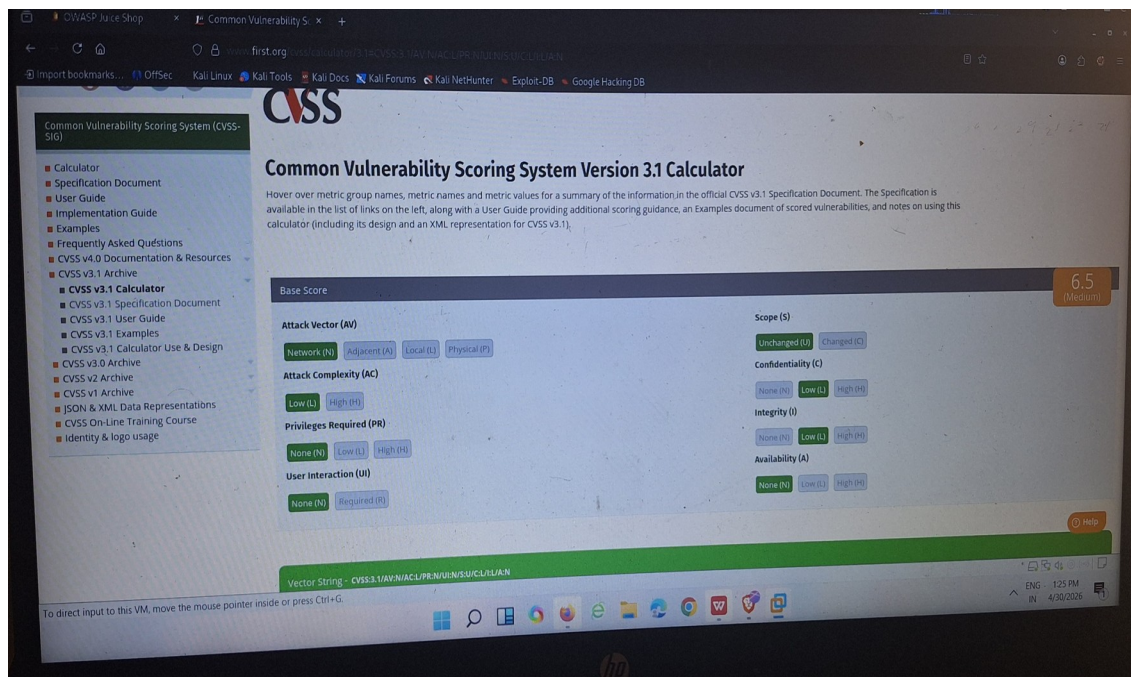


Figure 11: CVSS v3.1 Score 7.5 High — Brute Force No Lockout

Impact:

Confidentiality: HIGH — Any account can be compromised via automated password guessing

Integrity: LOW — Compromised accounts can modify data

Availability: NONE — No service disruption

Remediation:

- Lock account for 15 minutes after 5 failed attempts
- Implement CAPTCHA after 3 consecutive failures
- Add rate limiting — maximum 10 requests/minute per IP
- Send alert email to user on repeated failed attempts
- Log all failed authentication attempts

5.3 MEDIUM SEVERITY FINDINGS

Finding 4: Information Disclosure — Verbose Error Message

Severity: MEDIUM

CVSS Score: 5.3

CVSS Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

CWE: CWE-209: Information Exposure Through Error Messages

Location: <https://juice-shop.herokuapp.com/profile/image/file>

Discovery: April 30, 2026

Description :When an invalid file type is uploaded to the profile image endpoint, the application returns a detailed 500 error exposing internal server information including file paths, line numbers, and framework version.

Proof of Concept :

Action: Uploaded a .php file to profile image upload

Response:

- OWASP Juice Shop (Express ^4.22.1)
- 500 Error: Illegal file type at/app/build/routes/profileImageFileUpload.js:52:18 at process.processTicksAndRejections (node:internal/process/task_queue:104:5)

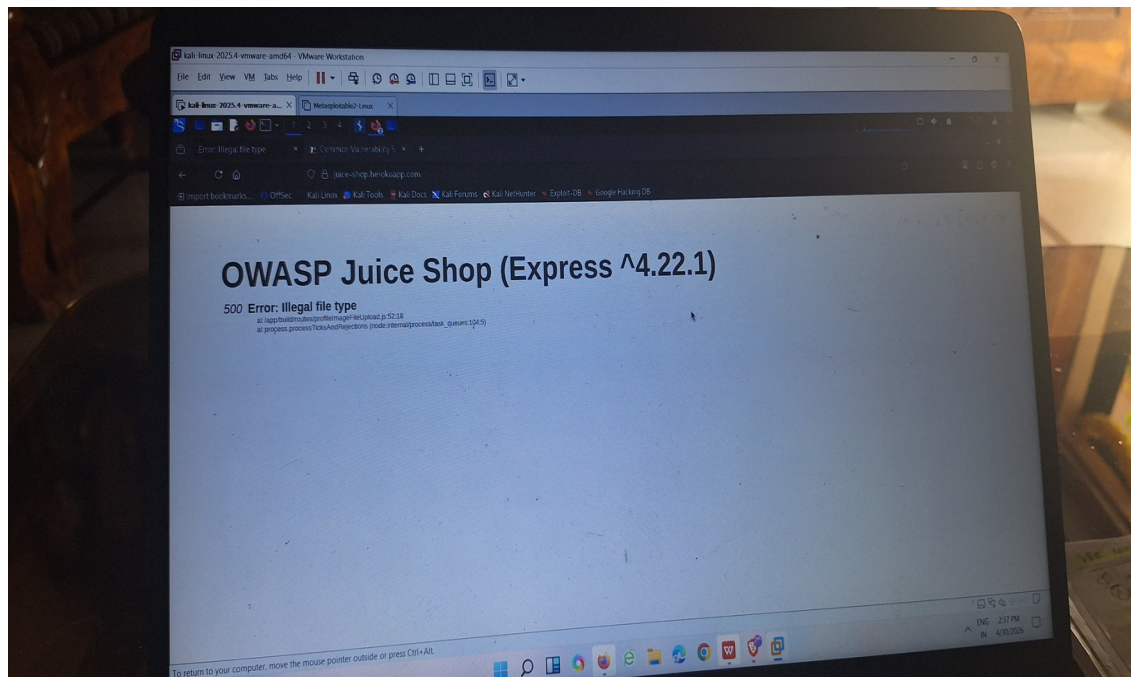


Figure 12: Information Disclosure — Verbose 500 error exposing internal file path and framework version

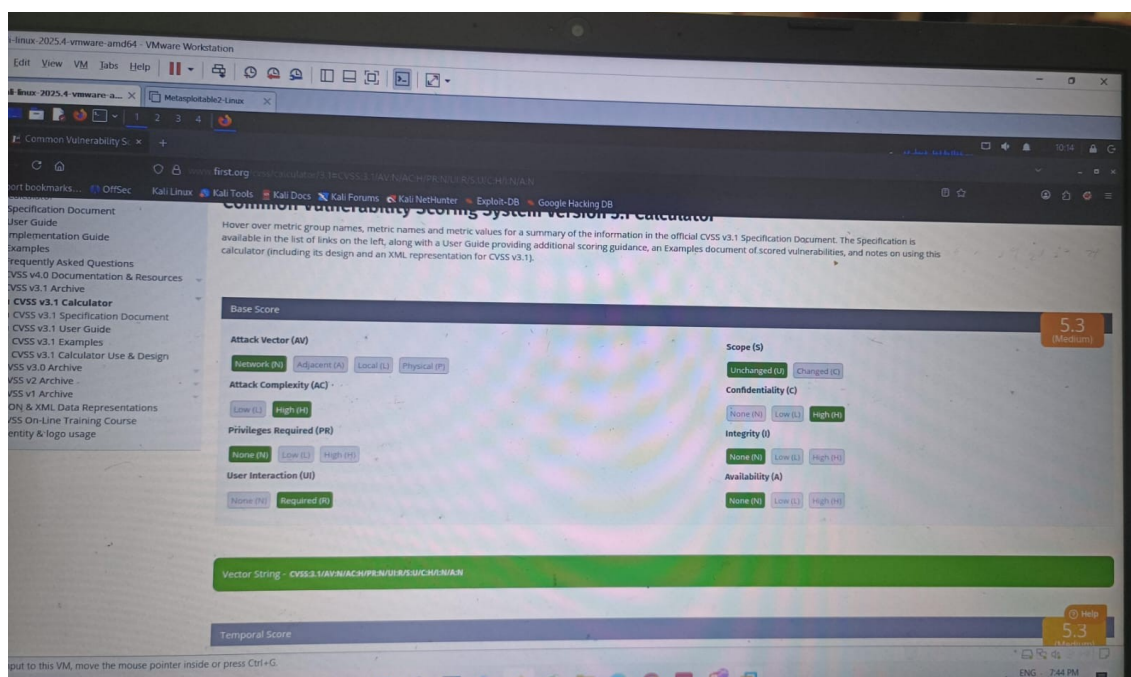


Figure 13: CVSS v3.1 Score 5.3 Medium — Information Disclosure

Impact:

- Internal file structure exposed to attacker
- Framework version revealed — enables targeted attacks
- Exact line numbers disclosed

Remediation: Immediate: Return only generic error messages to users: "An error occurred. Please try again."

Finding 5: Content Security Policy Header Missing

Severity: MEDIUM

CVSS Score: 5.4

CWE: CWE-693

Location: Application-wide

Discovered: OWASP ZAP automated scan

Description: The application does not implement a Content Security Policy (CSP) header. Without CSP, browsers cannot restrict which scripts are allowed to execute, making XSS attacks significantly easier to exploit.

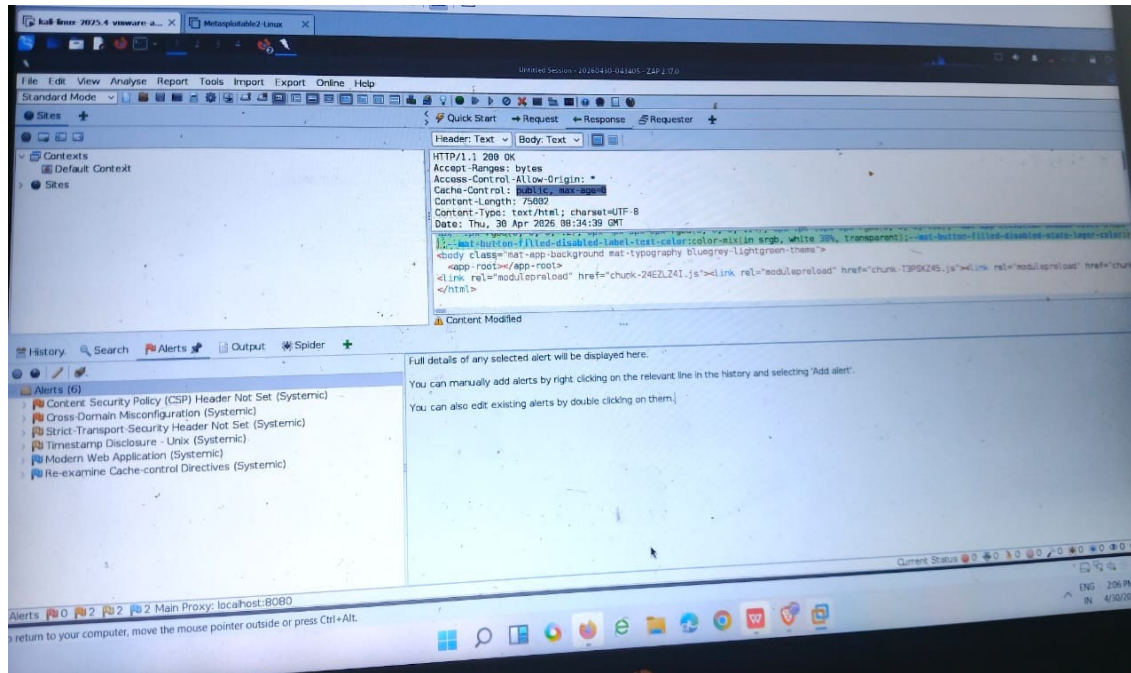


Figure 14: OWASP ZAP Automated Scan — 6 Alerts (CSP, Cross Domain, HSTS, Timestamp, Cache Control)

Remediation:

Add to all server responses: Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'

Finding 6: Cross Domain Misconfiguration

Severity: **MEDIUM**

CVSS Score: 5.3

CWE: CWE-942

Location: Application-wide

Discovered: OWASP ZAP automated scan

Description: The application allows unrestricted cross-domain access without proper CORS restrictions, potentially enabling unauthorized cross-origin requests from malicious sites.

Remediation:

Restrict CORS to trusted domains only

Access-Control-Allow-Origin: <https://trusteddomain.com>

Finding 7: HSTS Header Not Set

Severity: MEDIUM

CVSS Score: 4.8

CWE: CWE-319

Location: Application-wide

Discovered: OWASP ZAP automated scan

Description: HTTP Strict Transport Security (HSTS) header is missing. Without HSTS, browsers may connect over unencrypted HTTP, exposing user data to interception attacks.

Remediation:

Add this header to all responses: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

5.4 LOW SEVERITY FINDINGS

Finding 8: Timestamp Disclosure

Severity: LOW

CVSS Score: 3.1

CWE: CWE-200

Location: Application-wide

Discovered: OWASP ZAP automated scan

Description:

Server timestamps are exposed in HTTP response headers, revealing server time information which can assist attackers in timing-based attacks and server fingerprinting.

Remediation: Remove or suppress Date and timestamp headers from all server responses.

6. RISK ASSESSMENT MATRIX

Likelihood vs Impact Matrix:

Likelihood	Low Impact	Medium Impact	High Impact
High Likelihood	-	F8	F2 ,F3
Medium Likelihood	-	F5 ,F6,F7	F4
Low Likelihood	-		F1

Legend:

F1 = SQL Injection

F5 = CSP Missing

F2 = XSS Cookie Theft

F6 = Cross Domain Misconfiguration

F3 = Brute Force

F7 = HSTS Not Set

F4 = Info Disclosure

F8 = Timestamp Disclosure

Note: F1 (SQL Injection) is placed Low Likelihood as it requires knowledge of the injection point however CVSS remains 9.8 Critical due to zero authentication required and complete system compromise possible.

7. REMEDIATION TIMELINE

P0 — Immediate (0-24 hours):

Finding 1: SQL Injection

- Implement parameterized queries immediately
- Deploy WAF with SQL injection signatures

P1 — Urgent (1-7 days):

Finding 2: Reflected XSS

- Implement output encoding
- Add DOMPurify sanitization library

Finding 3: Brute Force

- Implement account lockout after 5 attempts
- Add CAPTCHA on login page

Finding 4: Information Disclosure

- Disable verbose error messages in production
- Implement generic error handler

P2 — Important (1-4 weeks):

Finding 5: Add Content Security Policy header

Finding 6: Restrict Cross Domain CORS policy

Finding 7: Enable HSTS header

P3 — Standard (1-3 months):

Finding 8: Suppress timestamp disclosure

Finding 9: Fix cache control directives

- Conduct follow-up penetration test to verify all fixes are effective

8. CONCLUSION AND RECOMMENDATIONS

The security assessment of OWASP Juice Shop identified 9 vulnerabilities ranging from Critical to Low severity. The most significant findings — SQL Injection and Cross-Site Scripting — were successfully exploited during testing, demonstrating real-world risk to user data and application integrity.

Key Recommendations:

1. Never trust user input — validate and sanitize all input server-side before processing
2. Implement security headers immediately — CSP, HSTS, and Cache-Control headers are simple one-line fixes with significant security impact
3. Use parameterized queries — eliminates SQL injection entirely regardless of input
4. Implement account lockout — prevents automated brute force attacks on all user accounts
5. Conduct regular penetration testing — every 6 months minimum, or after major code changes
6. Train developers on OWASP Top 10 — most vulnerabilities found are preventable with basic secure coding knowledge

Overall, the application requires immediate attention to Critical and High severity findings before any production deployment. Medium and Low findings should be addressed within the recommended timeline to achieve an acceptable security posture.

